

第三章 问题归约法与博弈

- 3.1 问题规约和与或图
- 3.2 盲目式搜索
- 3.3 机器博弈



3.1 问题归约表示和与或图

1. 引言
2. 问题归约法
3. 与或图

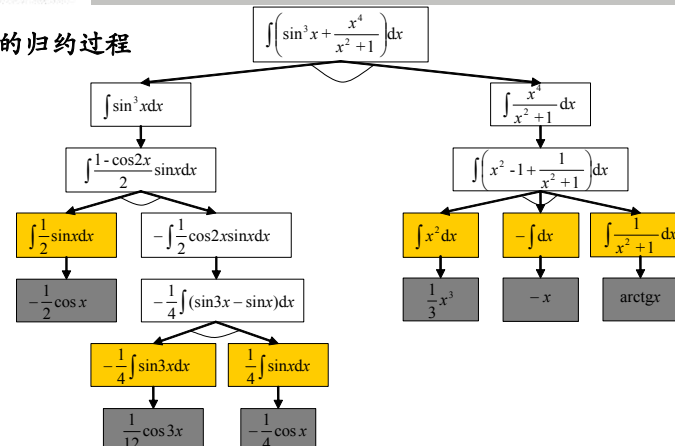


引言

- 状态空间表示: (S, F, G)
- 求解方法: 从问题的初始状态出发, 选择合适的算符系列、经过一系列的中间状态, 直到到达目标状态为止。

引言

不定积分的归约过程



➤ 对于一个复杂问题，直接求解往往比较困难。此时可通过下述方法进行简化：

• 分解

把一个复杂问题分解为若干个较为简单的子问题，每个子问题又可继续分解。重复此过程，直到不需要再分解或者不能再分解为止。

• 等价变换

利用同构或同态的等价变换，把原问题变换为若干个较为容易求解的新问题。

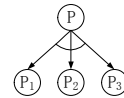


图3.1: 分解

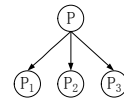


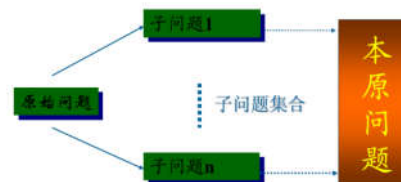
图3.2: 等价变换

➤ 问题归约是另一种问题描述与求解方法。所谓归约，即降阶。

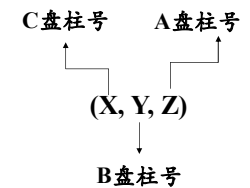
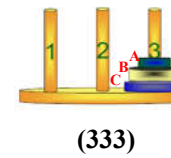
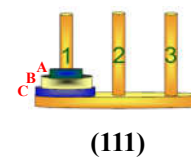
- **基本思想**：从已知问题的描述出发，通过一系列变换把此问题最终变为一个子问题集合；这些子问题的解可以直接得到，从而解决了初始问题。
- **问题规约的实质**：从目标(要解决的问题)出发逆向推理，建立子问题以及子问题的子问题，直至最后把初始问题归约为一个平凡的本原问题集合。
- 本原问题就是不可或不需再通过变换化简的“原子”问题，本原问题的解可以直接得到。

➤ 问题归约法的组成：

- (1)一个**初始问题描述**；
- (2)一套把问题变换为子问题的**操作符**；
- (3)一套**本原问题**；



- 有3个柱子(1,2,3)和3个不同尺寸的圆盘(A,B,C)。在每个圆盘的中心有个孔，所以圆盘可以堆叠在柱子上。
- 最初，全部3个圆盘都堆在柱子1上：最大的圆盘C在底部，最小的圆盘A在顶部。
- 要求把所有圆盘都移到柱子3上
 - 每次只许移动一个，而且只能先搬动柱子顶部的圆盘；
 - 还不许把尺寸较大的圆盘堆放在尺寸较小的圆盘上；





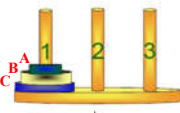
西安电子科技大学
XIDIAN UNIVERSITY

实例：三阶梵塔难题

➤ 原始梵塔问题可归约为较简单的3个子问题集合：

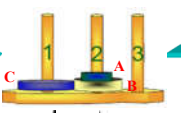
- 1. 移动A, B盘至2号柱子的双圆盘难题
- 2. 移动圆盘C至3号柱子的单圆盘问题
- 3. 移动A, B盘至3号柱子的双圆盘难题

(111)



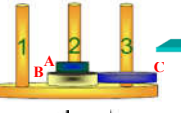
双圆盘难题

(122)



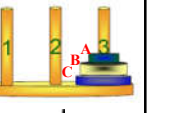
本原问题

(322)



双圆盘难题

(333)



人工智能学院

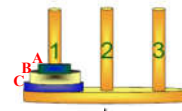


西安电子科技大学
XIDIAN UNIVERSITY

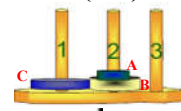
实例：三阶梵塔难题

➤ 双圆盘难题（一）

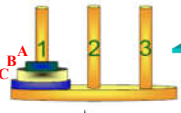
(111)



(122)

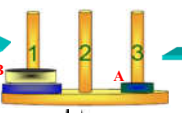


(111)



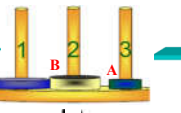
本原问题

(122)



本原问题

(133)



本原问题

人工智能学院

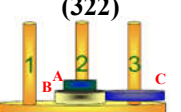


西安电子科技大学
XIDIAN UNIVERSITY

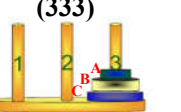
实例：三阶梵塔难题

➤ 二阶梵塔难题（二）

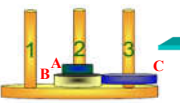
(322)



(333)

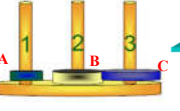


(111)



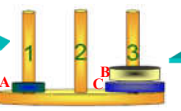
本原问题

(122)



本原问题

(133)



本原问题

人工智能学院

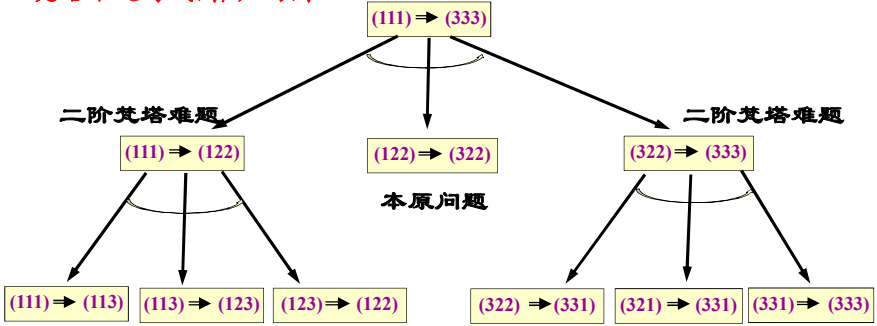


西安电子科技大学
XIDIAN UNIVERSITY

实例：三阶梵塔难题

➤ 梵塔难题与或图/归约图

(111) ⇒ (333)



二阶梵塔难题

本原问题

二阶梵塔难题

本原问题

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

问题规约法

➤ 问题归约法的基本思路是：应用一系列算符将原始问题的描述变换或分解成为子问题的描述。

➤ 问题的描述可以采用各种数据结构，如表、树、矢量、数组等

对于梵塔问题，原始问题：(111)→(333)

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

问题规约法

问题归约法可以用一个三元组 (S, O, P) 来表示，其中：

➤ S：原始问题，即要解决的问题

➤ P：本原问题集，其中的每一个问题是不用证明的或自然成立的，例如公理、已知事实等

➤ O：操作算子集，用于将问题化为子问题

人工智能学院



西安电子科技大学

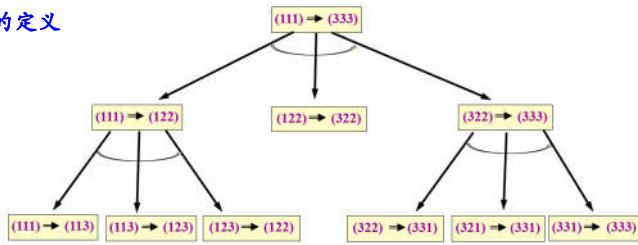
XIDIAN UNIVERSITY

与或图表示

➤ 对归约问题，可采用图的结构来表示把问题归约为子问题的替换集合。

➤ 表示问题规约的图称为与或图（归约图）

- 与或图的结构
- 节点可解性的定义



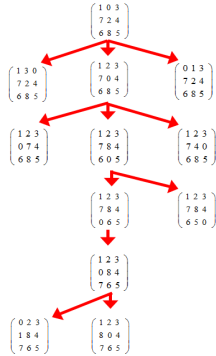
人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或图结构



三阶梵塔的和或图

八数码难题的部分状态空间图

人工智能学院

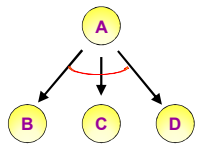


西安电子科技大学
XIDIAN UNIVERSITY

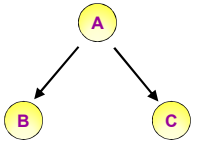
与或图结构

➤ 例如，设想问题A需要由求解问题B、C和D来决定，那么可以用一个与图来表示(左图);

➤ 同样，一个问题A或者由求解问题B、或者由求解问题C来决定，则可以用一个或图来表示(右图);



与图



或图

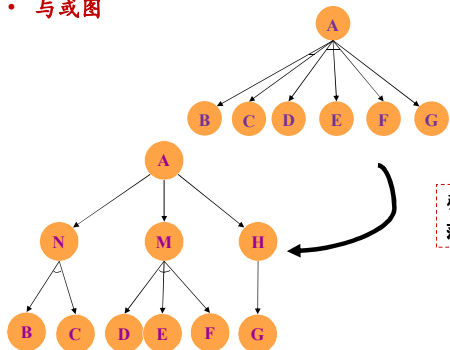
人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

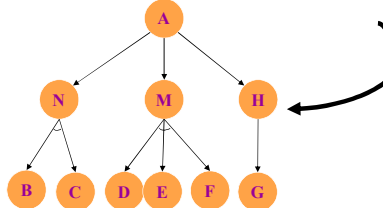
与或图结构

• 与或图



考虑某规约问题A:

- 即可由求解问题B和C来解决
- 也可由求解问题D、E和F来解决
- 或者由单独求解G来解决



引入中间节点N和M，可使图更规范，更容易被计算机存储与处理

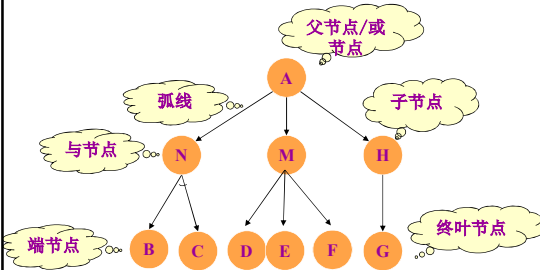
其中标记为N和M的附加节点分别作为集合{B, C}和{D, E, F}的唯一父节点，具有辅助问题描述的作用。

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

与或图的一些术语



➤ 如果某条弧线从节点a指向节点b，那么节点a叫做节点b的父节点；节点b叫做节点a的后继节点或子节点；

➤ 或节点：一个节点的子节点是或的关系，称其为或节点。

➤ 与节点：一个节点的子节点是与的关系，称其为与节点。

➤ 终叶节点：是对应于原问题的本原节点。

➤ 端节点：与或图中除终叶节点外无子节点的节点。

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

与或图特例

➤ 所有节点都是或节点，这时就是一般的图，即状态空间图

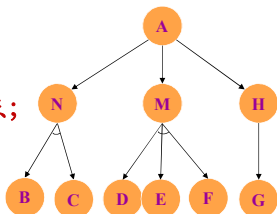
- 状态空间搜索问题与与或图问题的区别在于其不存在任何与节点。
- 由于与或图出现与节点，其结构与其他图结构大为不同。因此，与或图需要有其特有的搜索技术，而且是否存在与节点也就成为区别两种问题求解方法的主要依据。

➤ 除了起始节点外，所有节点只有一个父节点，此时称为与或树。

人工智能学院

➤ 与或图的结构

- 初始节点对应于原始问题描述;
- 对应于本原问题的节点叫做终叶节点/叶节点;
- 中间问题对应非终叶节点;



在与或图中, 问题有解的条件是: 起始节点是可解的。

➤ 可解节点 (递归定义):

- 终叶节点是可解节点 (因为它们直接和本原问题相关连);
- 对于或节点, 只有当其后继节点至少有一个是可解的时, 此节点才是可解的;
- 对于与节点, 那么只有当其后继节点全部为可解时, 此节点才是可解的。

➤ 不可解节点:

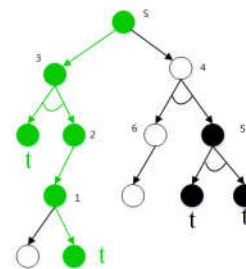
- 没有后继节点的非终叶节点为不可解节点;
- 对于或节点, 全部后继节点为不可解, 此节点才是不可解的;
- 对于与节点, 后继节点至少有一个为不可解的, 此节点才是不可解的;

➤ 一个节点是可解, 则节点须满足下列条件之一:

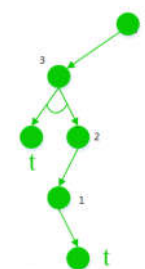
- 1、终止节点是可解节点;
- 2、一个与节点可解, 当且仅当其子节点全都可解;
- 3、一个或节点可解, 只要其子节点至少有一个可解;

➤ 一个节点是不可解, 则节点须满足下列条件之一:

- 1、非终叶节点的端节点是不可解节点;
- 2、一个与节点不可解, 只要其子节点至少有一个不可解;
- 3、一个或节点不可解, 当且仅当其子节点全都不可解。



与或树



解树

图中可解节点用实圆圈表示, 不可解节点用圆圈表示

西安电子科技大学

XIDIAN UNIVERSITY

可解节点与不可解节点

节点是否可解通过由下而上进行的可解标志过程来确定。

- 可解/不可解标志：根据可解节点与不可解节点的递归定义，通过递归的方式作用于某个与或图上，以标出所有的可解节点及不可解节点。
- 一个解树被定义为那些可解节点所构成的子图，这些节点能够证明问题的初始节点是可解的。
- 解树是绿色子图

图3.3

图3.4

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

问题归约与状态空间

方法	初始问题	算符	目标	结果
状态空间	状态	算符或操作	目标状态	解路径
问题归约	问题	分解或变换	初始问题可解	解树

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

第三章 问题归约法与博弈

- 3.1 问题规约和与或图
- 3.2 盲目式搜索
- 3.3 机器博弈

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

3.2 与或树的盲目式搜索

- 与或树搜索
- 与或树的宽度优先搜索
- 与或树的深度优先搜索

人工智能学院

➤ 与或图搜索：在与或图上执行搜索的过程，其目的在于标明起始节点是有解的，即搜索不是去寻找目标节点的一条路径，而是寻找一个解树。

- 执行可解节点标志和不可解节点标志过程。
- 解树：由能够证明初始节点是可解的可解节点构成的连通的子图。

➤ 与或图/与或树

- 与或树：除初始节点，其余节点只有一个父节点
- 与或图：除初始节点，其余节点允许有多个父节点

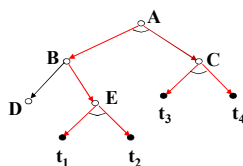


图3.5：与或树

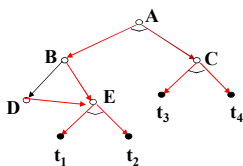


图3.6：与或图

➤ 与或树的一般搜索过程：

- (1) 把原始问题作为初始节点 S_0 ，并把它作为当前节点；
- (2) 应用分解或等价变换对当前节点进行扩展；
- (3) 为每个节点设置指向父节点的指针；
- (4) 选择适合的节点作为当前节点，反复执行第(2)步和第(3)步，在此其间要多次调用可解标志过程和不可解标志过程，直到初始节点被标为可解节点或不可解节点为止。

➤ 与或树的搜索的目的是寻找解树，从而求得原始问题的解。

➤ 与或树搜索与状态空间图搜索的区别：

- 搜索目的不同：是证明起始节点是否可解，而可解节点是递归定义的，取决于后继节点是否可解。
- 结果不同：若初始节点被标志为可解，则搜索成功结束；若初始节点被标志为不可解，则搜索失败。
- 节点处理不同：一旦发现不可解节点，应把该节点从图中删去。

与/或树的搜索策略

- 盲目式搜索
 - 宽度优先搜索
 - 深度优先搜索
 - 等代价搜索
- 启发式搜索



西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索

➤ 与或树的宽度优先搜索的基本思想:

- 按照“先产生的节点先扩展”的原则进行搜索;
- 搜索过程中要多次调用可解标志过程和不可解标志过程;
- OPEN表、CLOSED表类似状态空间搜索;
 - OPEN表: 存放待扩展的节点
 - CLOSED表: 存放已扩展的节点

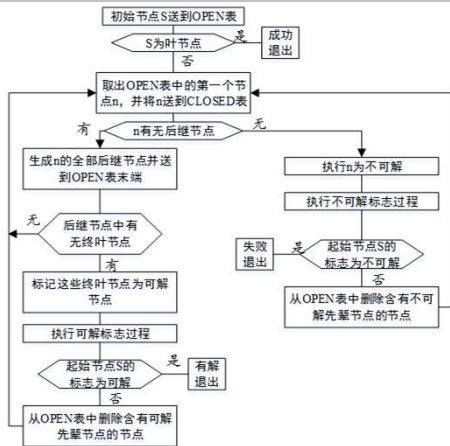
人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索流程



```

graph TD
    Start([初始节点S送到OPEN表]) --> IsLeaf{S为叶节点}
    IsLeaf -- 是 --> Success([成功退出])
    IsLeaf -- 否 --> GetNode[取出OPEN表中的第一个节点n, 并将n送到CLOSED表]
    GetNode --> HasChild{ n有后继节点 }
    HasChild -- 有 --> GenChild[生成n的全部后继节点并送到OPEN表末端]
    GenChild --> NoChild{后继节点中有无叶节点}
    NoChild -- 有 --> MarkSolvable[标记这些叶节点为可解节点]
    MarkSolvable --> ExecSolvable[执行可解标志过程]
    ExecSolvable --> IsSolvable{起始节点S的标志为可解}
    IsSolvable -- 是 --> ExitSolvable([有解退出])
    IsSolvable -- 否 --> DeleteSolvable[从OPEN表中删除含有可解先辈节点的节点]
    DeleteSolvable --> GetNode
    NoChild -- 无 --> NoChildSolvable{后继节点中无叶节点}
    NoChildSolvable --> ExecUnsolvable[执行n为不可解]
    ExecUnsolvable --> ExecUnsolvableProc[执行不可解标志过程]
    ExecUnsolvableProc --> IsUnsolvable{起始节点S的标志为不可解}
    IsUnsolvable -- 是 --> ExitUnsolvable([失败退出])
    IsUnsolvable -- 否 --> DeleteUnsolvable[从OPEN表中删除含有不可解先辈节点的节点]
    DeleteUnsolvable --> GetNode
    
```

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索流程

- 起始节点S送OPEN表
- 若S为叶节点, 则成功结束, 否则, 继续
- 取出OPEN表的第一个节点(记作n), 并送到CLOSED表
- 扩展节点n, 生成其全部后继节点, 送OPEN表末端, 并设置指向n的指针
说明: 此时可能出现三种情况
 - 节点n 无后继节点
 - 节点n 有后继节点、并有叶节点
 - 节点n 有后继节点、但无叶节点
- 若n 无后继节点, 标志n 为不可解, 并转9 (10、11);
 - 若后继节点中有叶节点, 则标志这些叶节点为可解节点, 并继续6、7、8
 - 否则转第3步
- 实行可解标志过程
- 若起始节点S标志为可解, 则找到解而结束, 否则继续
- 从OPEN表中删去含有可解先辈节点的节点, 并转第3步
- 实行不可解标志过程
- 若起始节点S标志为不可解, 则失败而结束, 否则继续
- 从OPEN表中删去含有不可解先辈节点的节点
- 转第3步

人工智能学院

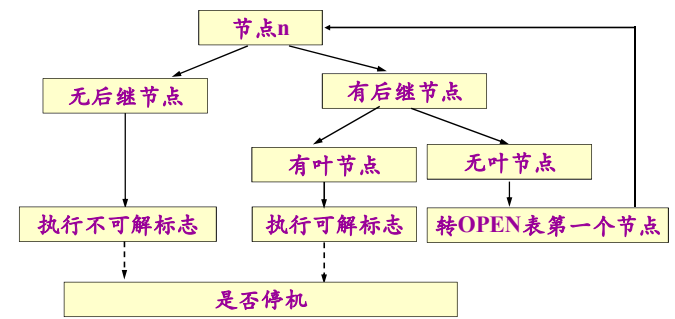


西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索

➤ 关键步骤: 扩展节点n, 生成其全部后继节点, 送到OPEN表末端, 并设置指向n的指针。



```

graph TD
    NodeN[节点n] --> NoChild[无后继节点]
    NodeN --> HasChild[有后继节点]
    NoChild --> ExecUnsolvable[执行不可解标志]
    HasChild --> HasLeaf[有叶节点]
    HasChild --> NoLeaf[无叶节点]
    HasLeaf --> ExecSolvable[执行可解标志]
    NoLeaf --> TurnNext[转OPEN表第一个节点]
    ExecUnsolvable -.-> IsStop[是否停机]
    ExecSolvable -.-> IsStop
    TurnNext --> NodeN
    
```

人工智能学院



西安电子科技大学
 XI'DIAN UNIVERSITY

与或树的宽度优先搜索

➤ 算法结束的条件

- 若初始节点被标示为可解节点，算法成功结束（有解）
- 若初始节点被标示为不可解结点，则搜索失败结束（无解）

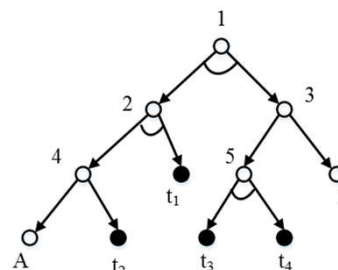
人工智能学院



西安电子科技大学
 XI'DIAN UNIVERSITY

与或树的宽度优先搜索

例1：设有如图所示的与/或树，其中 t_1, t_2, t_3, t_4 均为终叶节点，A和B是不可解的端节点。用宽度优先搜索法求出解树。



人工智能学院



西安电子科技大学
 XI'DIAN UNIVERSITY

与或树的宽度优先搜索

➤ 初始化：

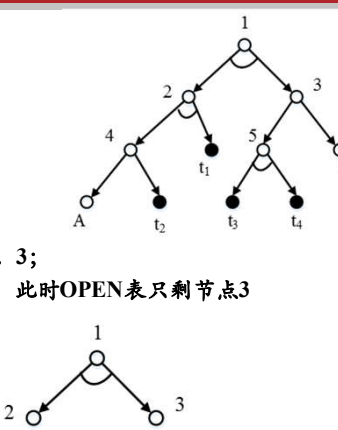
节点1送到OPEN表，且不为终叶节点

OPEN表	CLOSED表
1	

➤ STEP1:

- 把节点1放入CLOSED表, 扩展，得到节点2、3；
- 节点2、3都不是终叶节点，接着扩展节点2，此时OPEN表只剩节点3

OPEN表	CLOSED表
1	
2,3	1
3	1,2



人工智能学院



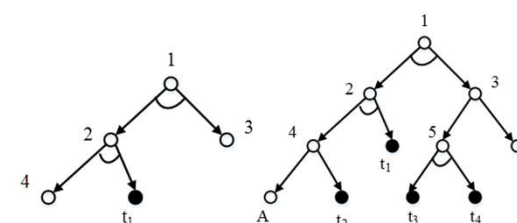
西安电子科技大学
 XI'DIAN UNIVERSITY

与或树的宽度优先搜索

➤ STEP2:

- 扩展节点2后，得到节点4、 t_1 ，此时OPEN表中的节点有3、4、 t_1 （***有顺序）；
- 节点 t_1 是终叶节点且为可解节点，对其先辈节点进行标志；
- t_1 的父节点是“与”节点，无法判断节点2是否可解，接着扩展节点3；

OPEN表	CLOSED表
1	
2,3	1
3,4, t_1	1,2
4, t_1	1,2,3



人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索

STEP3:

- 扩展节点3后，得到节点5、B；
- 节点5，B都不是终叶节点，接着扩展节点4；

OPEN表	CLOSED表
1	
2,3	1
3,4,t ₁	1,2
4,t ₁ ,5,B	1,2,3
t ₁ ,5,B	1,2,3,4

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索

STEP4:

- 扩展节点4后，得到节点A、t₂；
- 节点t₂是终叶节点且为可解节点，对其先辈节点进行标志；
- 节点4、2可解，但不确定1是否可解；从OPEN表中删掉A
- t₁直接移到CLOSED表
- 此时节点5是OPEN表中第一个待考察的节点，下一步将扩展节点5

OPEN表	CLOSED表
1	
2,3	1
3,4,t ₁	1,2
4,t ₁ ,5,B	1,2,3
t ₁ ,5,B,A,t ₂	1,2,3,4
t ₁ ,5,B,A,t ₂	1,2,3,4
5,B,t ₂	1,2,3,4,t ₁

说明：OPEN表中的终叶节点直接移到CLOSED表中，不做处理

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索

STEPS:

- 扩展节点5后，得到节点t₃、t₄；
- 节点t₃、t₄都是终叶节点，且为可解节点，对其先辈节点进行标志；
- 节点5可解，节点3可解，节点1可解，成功退出。

说明：OPEN表中的终叶节点直接移到CLOSED表中，不做处理

OPEN表	CLOSED表
1	
2,3	1
3,4,t ₁	1,2
4,t ₁ ,5,B	1,2,3
t ₁ ,5,B,A,t ₂	1,2,3,4
t ₁ ,5,B,A,t ₂	1,2,3,4
5,B,t ₂	1,2,3,4,t ₁
B,t ₂ ,t ₃ ,t ₄	1,2,3,4,t ₁ ,5

图3.7：解树

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的宽度优先搜索

“先产生的节点先被扩展”

如果问题有解，宽度优先搜索一定会求得一个解树。

- 初始节点深度为零
- 其他节点的深度是其父节点深度加1

与或树

宽度优先搜索的解树

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索

➤ 与或树的深度优先搜索

- 按照“**新产生的节点先扩展**”的原则进行搜索
- 设置深度界限
- 搜索过程中要多次调用可解标志过程和不可解标志过程

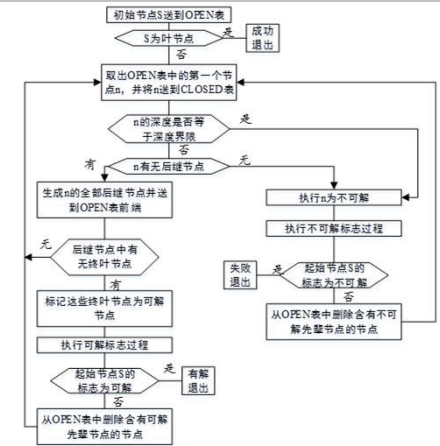
人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索流程



```

graph TD
    Start([初始节点S送入OPEN表]) --> IsLeaf{S为叶节点}
    IsLeaf -- 是 --> Success([成功退出])
    IsLeaf -- 否 --> GetNode[取出OPEN表中的第一个节点n, 并将n送入CLOSED表]
    GetNode --> DepthLimit{n的深度是否等于深度界限}
    DepthLimit -- 是 --> NoSolvable[执行n为不可解]
    DepthLimit -- 否 --> HasChildren{n有无后继节点}
    HasChildren -- 有 --> GenerateChildren[生成n的全部后继节点并送到OPEN表前端]
    GenerateChildren --> AllLeaves{后继节点中  
有无线叶节点}
    AllLeaves -- 有 --> MarkSolvable[标记这些叶节点为可解节点]
    MarkSolvable --> ExecSolvable[执行可解标志过程]
    ExecSolvable --> StartSolvable{起始节点S的  
标志为可解}
    StartSolvable -- 是 --> SuccessSolvable([有解退出])
    StartSolvable -- 否 --> DeleteSolvable[从OPEN表中删除含有可解  
先辈节点的节点]
    DeleteSolvable --> GetNode
    AllLeaves -- 无 --> NoChildren[无]
    NoChildren --> NoSolvable
    NoSolvable --> ExecNoSolvable[执行不可解标志过程]
    ExecNoSolvable --> MarkNoSolvable{起始节点S的  
标志为不可解}
    MarkNoSolvable -- 是 --> SuccessNoSolvable([失败退出])
    MarkNoSolvable -- 否 --> DeleteNoSolvable[从OPEN表中删除含有不可  
解先辈节点的节点]
    DeleteNoSolvable --> GetNode
  
```

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索流程

- 起始节点S送入OPEN表
- 若S为叶节点, 则成功结束, 否则继续
- 取出OPEN表第一个节点 (记作n), 送到CLOSED表
- 若节点n的深度等于深度界限, 则将n标志为不可解节点, 并转(10); 否则继续
- 扩展节点n, 生成全部后继节点, 置于OPEN表的前面, 并设置指向n的指针
- 分三种情况
 - 如果n无后继节点, 则标志为不可解节点, 并转(10), 否则继续
 - 若有后继节点为叶节点, 则将这些叶节点标志为可解节点, 并继续;
 - 否则转(3).
- 实行可解标志过程
- 若起始节点为可解节点, 则算法成功结束; 否则, 继续下一步
- 从OPEN表中删除含有可解先辈节点的节点, 并转(3)
- 实行不可解标志过程
- 若起始节点为不可解, 则失败结束, 否则, 继续下一步
- 从OPEN表中删去含有不可解先辈节点的节点
- 转(3)

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索

➤ 与宽度优先算法相比, 深度优先算法的特殊之处:

- 第4步: 要判断从OPEN表取出来的节点的深度。如果等于深度界限, 认定它为不可解节点;
- 第6步: 扩展节点n把其子节点放入OPEN表的前端, 即**新产生的节点先扩展**;

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索

例2：设有如图所示的与/或树，其中 t_1, t_2, t_3, t_4 均为终叶节点，A和B是不可解的端节点。采用深度优先搜索法求出解树。（规定深度界限为4）

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索

初始化：

节点1 送到OPEN表，且不为终叶节点

OPEN表	CLOSED表
1	

STEP1:

- 扩展节点1，得到节点2、3；
- 节点2、3都不是终叶节点，接着扩展节点2.

OPEN表	CLOSED表
1	
2,3	1
3	1,2

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索

STEP2:

- 扩展节点2后，得到节点4、 t_1 ；
- t_1 是终叶节点，但无法标识节点2；
- 继续扩展节点4；

OPEN表	CLOSED表
1	
2,3	1
4, t_1 , 3	1,2
t_1 , 3	1,2,4

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

与或树的深度优先搜索

STEP3:

- 扩展节点4后，得到节点A、 t_2 ；
- 标志4、2为可解节点，删掉OPEN中的A，不能确定1是否可解
- 继续扩展3

OPEN表	CLOSED表
1	
2,3	1
4, t_1 , 3	1,2
A, t_2 , t_1 , 3	1,2,4
t_2 , t_1 , 3	1,2,4
3	1,2,4, t_2 , t_1

人工智能学院

西安电子科技大学
 XIDIAN UNIVERSITY

与或树的深度优先搜索

➤ STEP4:

- 扩展节点3后, 得到节点5、B;
- 接着扩展节点5;

OPEN表	CLOSED表
1	
2,3	1
4,t ₁ ,3	1,2
A,t ₂ ,t ₁ ,3	1,2,4
3	1,2,4,t ₁
5,B	1,2,4,t ₂ ,t ₁ ,3
B	1,2,4,t ₂ ,t ₁ ,3,5

人工智能学院

西安电子科技大学
 XIDIAN UNIVERSITY

与或树的深度优先搜索

➤ STEP5:

- 扩展节点5后, 得到节点t₃,t₄;
- 标志可解节点5, 3, 得出初始节点1可解,;
- 成功退出。

OPEN表	CLOSED表
1	
2,3	1
4,t ₁ ,3	1,2
A,t ₂ ,t ₁ ,3	1,2,4
t ₂ ,t ₁ ,3	1,2,4
5,B	1,2,4,t ₂ ,t ₁ ,3
t ₃ ,t ₄ ,B	1,2,4,t ₂ ,t ₁ ,3,5

人工智能学院

西安电子科技大学
 XIDIAN UNIVERSITY

与或树的深度优先搜索

与或树

深度优先搜索的解树

人工智能学院

西安电子科技大学
 XIDIAN UNIVERSITY

小结

➤ 与或树的宽度优先搜索算法

- 要点: 先产生的节点先被扩展

➤ 与或树的深度优先搜索

- 要点: 新产生的节点先被扩展

➤ 其他盲目式搜索算法:

- 等代价搜索

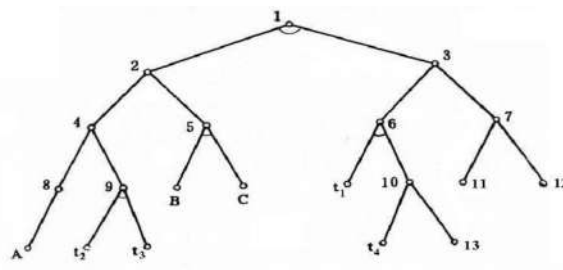
人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

练习

设有如下所示的与或树，请分别用与或树的宽度优先搜索和深度优先搜索求解树。



人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

人工智能概论

主讲：李建霞

1761389691@qq.com

西安电子科技大学 人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

第三章 问题归约法与博弈

- 3.1 问题规约和与或图
- 3.2 盲目式搜索
- 3.3 机器博弈



人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

3.3 博弈树搜索

- 1. 机器博弈
- 2. Max-Min搜索
- 3. α - β 剪枝



人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

引言

➤ 你可曾听说过“深蓝”？

1997年5月11日，IBM开发的“**深蓝**”击败了国际象棋冠军卡斯帕罗夫。

➤ 卡斯帕罗夫是何许人也？

- 6岁下棋
- 13岁获得全苏青年赛冠军
- 16岁获世界青年赛第一名
- 1980年获得世界少年组冠军（17岁）
- 1982年并列夺得苏联冠军（19岁）
- 1985年成为**历史上最年轻**的国际象棋冠军（22岁）



卡斯帕罗夫（1963-）

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

引言

➤ 深蓝的前身

- 1988年“**深思**”击败了丹麦特级大师拉森。
- 1993年“**深思**”第二代击败了丹麦世界优秀女棋手小波尔加。

➤ 深蓝的改进版

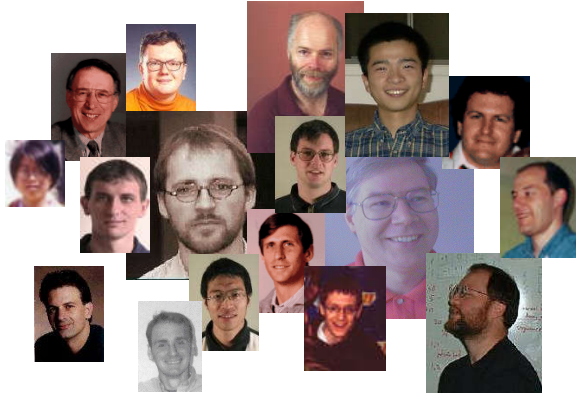
- 2001年“**弗里茨**”击败了除了克拉姆尼克之外的所有排名世界前十位的棋手。
- 2002“**更弗里茨**”与世界棋王克拉姆尼克在巴林交手，双方以4比4战平。
- 2003年“**更年少者**”与卡斯帕罗夫在纽约较量，3比3战平。

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

引言



电脑旗手永不停歇的挑战在人类，而人类不甘失败，世界各地的人类旗手仍在努力。人机大战的一直在进行！

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

机器博弈历史

➤ 国际象棋：

- 1958年，第一台象棋计算机“思考”诞生
- 改进版本：深思、深思改进版、深蓝、更深的蓝、弗里茨以及更弗里茨等

➤ 西洋跳棋：

- 塞缪尔1959年研制的西洋跳棋程序战胜塞缪尔本人，1962年又击败了当时的西洋棋选手罗伯特
- 斯卡费尔的2016年的切努克终极程序：
1989年开始，直至2016年斯卡费尔的切努克终极程序完成，并宣布在西洋跳棋上，人类已经不可能战胜计算机程序。

➤ 西洋双陆棋：

- 汉斯·伯林1979年开发了西洋双陆棋的强程序：
以7:1的成绩击败了当时的世界冠军。创下了计算机第一次在智力游戏中击败冠军级人类竞争对手的历史。

人工智能学院

机器博弈：历史渊源留长

➤ 德州扑克:

- 2017年，美国卡耐基梅隆大学开发的德州扑克博弈系统 Libratus：在与 4 名人类顶尖扑克选手的人机大战中获得了胜利
- 2019人工智能 Pluribus：在六人无限注德州扑克比赛中击败多名世界顶尖选手，成为机器在多人游戏中战胜人类的一个里程碑。

➤ 中国围棋 (10^{172}) :

- 2017年，AlphaGo与排名世界第一的世界围棋冠军柯洁对战，以3比0的总比分获胜。

➤ 中国桥牌、星际争霸、DOTA2等许多种其它种类游戏博弈的研究也正在进行中。

机器博弈的历史是人工智能发展的缩影，每一次成功都代表着人工智能的一次进步。

► 博弈：智能竞技

- 本意：下棋
- 【出处】：《论语·阳货》：“**饱食终日，无所用心**，难矣哉！不有**博弈者乎**？为之，犹贤乎已。”
- 实例：一字棋、象棋、围棋、五子棋、跳棋（最多六人）等。
- 有双方博弈和多方博弈之分
- 本节只考虑二人博弈（双方博弈），特点：二人对抗，轮流操作，一方获胜或战平停止



➤ **机器博弈**：机器参与的博弈，参与智力竞技。

- **博弈的目标是取胜**，取胜的棋局如同状态空间法中的目标状态。
- 与八数码游戏一样，参与者需要对棋局进行操作，以改变棋局，使其向目标棋局转移。
- **八数码游戏只涉及一个主体，不是博弈。**

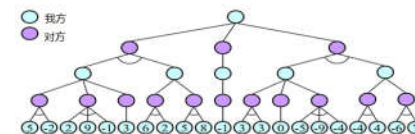


➤ 如何根据当前的棋局，选择对自己最有利的一步棋？

► 博弈的问题表示：博弈树表示

- 一种特殊的与或树。
- 节点：博弈的格局（即棋局），相当于状态空间中的状态，反映了博弈的信息，并且与节点、或节点隔层交替出现。

博弈树搜索技术一直以来是机器博弈中的基础技术，尤其在早期的人工智能棋类游戏中占据主要地位。



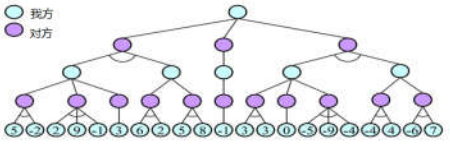


西安电子科技大学
XIDIAN UNIVERSITY

博弈树

➤ 与节点与或节点为什么交替出现呢？

- 博弈双方为：我方和对方
- 在博弈过程中，规则是双方**轮流走步**。在博弈树中，相当于博弈双方轮流扩展其所属节点



人工智能学院

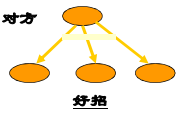


西安电子科技大学
XIDIAN UNIVERSITY

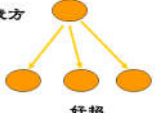
博弈树

➤ 从我方的角度来看：

对方



我方



- **所有对方节点都是与节点**
理由：对于轮到对方走棋时，对于我方来说，必须能够应付对手的每一种走法，这就相当于这些棋是“与”的关系。
- **所有属于我方的节点都是“或节点”**
理由：当轮到我方走棋时，只需从若干个可以走的棋中，选择一个走法走就可以了。从这个意义上说，若干个可以走的棋是“或”关系。

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

博弈树

➤ 描述博弈过程的与或树为博弈树，特点为：

- 博弈的初始格局是初始节点；
- 博弈树中，或节点与与节点逐层交替出现，自己一方扩展节点是“或”关系，对方扩展节点是“与”关系；
- 所有能使自己一方获胜的终局是本原问题，相应节点为可解节点，所有使对方获胜的终局都是不可解节点

➤ 博弈树搜索

- 极大极小 (Max-Min) 搜索
- α - β 剪枝

人工智能学院



西安电子科技大学
XIDIAN UNIVERSITY

Max-Min搜索

➤ 基本思想：

- 目的是为博弈的双方中的一方寻找一个最优行动方案；
- 要寻找这个最优方案，就要通过计算当前所有可能的方案来进行比较；
- 方案的比较是根据问题的特征来定义一个估价函数，用来估算**当前博弈树端节点**的得分；
- 当计算出端节点的估值后，再推算出父节点的得分（即计算倒推值）；
 - 1) 对或节点，选其子节点中一个最大得分作为其得分
 - 2) 对与节点，选其子节点中一个最小得分作为其得分
- 如果一个行动方案能获得较大的倒推值，则它就是当前最好的行动方案。

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索

Step1. 生成k-步博弈树

- Max 代表机器一方 / Min 代表敌方
- 设 Max 面对的当前棋局为 $c^{(0)}$, 以 $c^{(0)}$ 为根, 生成 k-步博弈树:

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索

Step2. 评估棋局(博弈状态)

- 估价函数
 - 为特定的博弈问题定义一个估价函数 $est(c)$, 用以评估 k-步博弈树叶节点对应的棋局 c
 - $est(c)$ 的值越大, 意味着棋局 c 对 Max 越有利。

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索

Step3. 回溯评估

- 极大极小运算

由叶节点向根节点方向回溯评估, 在 Max 处取最大评估值(或运算), 在 Min 处取最小评估值(与运算)。

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索

Step3. 回溯评估

- 极大极小运算

- Max 按取最大评估值的方向行棋

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索

Step4. 递归循环

- Max 行棋后，等待 Min 行棋;
- Min 行棋后，即产生对于 Max 而言新的当前棋局 $c^{(0)}$;
- 返回 Step1，开始下一轮博弈。

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索

Max-Min搜索：两个阶段：

- 第一阶段：**用宽度优先算法生成规定深度 k 的全部博弈树，然后对其所有端节点计算 $est(c)$ 。
- 第二阶段：**是自下而上逐级求节点的倒推估价值，直至求出初始节点的 $est(c^{(0)})$ 为止，再由 $est(c^{(0)})$ 选出相对较好的走法，过程结束。

对手走出相应的棋，再以当前的格局作为初始节点，重复此过程，选择对自己有利的走法。

人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

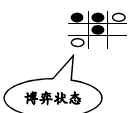
Max-Min搜索举例

一字棋：

设有 3×3 棋格，Max 与 Min 轮流行棋，黑先白后，先将 3 颗棋子连成一线的一方获胜。

- 一字棋博弈空间：**共有 9! 种可能的博弈状态
- 一字棋算子空间：**博弈规则集合
- 一字棋博弈目标集合(对 Max而言, 15):**

$$\sum c^{(g)} = \left\{ \begin{matrix} \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \circ & \circ & \circ \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \circ & \circ & \circ \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} & \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array} \end{matrix} \right\}$$



人工智能学院



西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索举例

一字棋：

- 定义估价函数： $est(c)$**

(1) 对于非终局的博弈状态 c ，估价函数为：

$$est(c) = (\text{所有空格都放上黑色棋子之后, 3颗黑色棋子连成的直线总数}) - (\text{所有空格都放上白色棋子之后, 3颗白色棋子连成的直线总数})$$

例如: $c1 = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \circ \\ \hline \bullet & \bullet & \circ \\ \hline \circ & \circ & \circ \\ \hline \end{array}$ $est(c1) = 3 - 2 = 1$

(2) 若 $c2$ 是 Max 的胜局，则：

$$est(c2) = +\infty$$

例如: $c2 = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \circ & \circ & \bullet \\ \hline \end{array}$

(3) 若 $c3$ 是 Min 的胜局，则：

$$est(c3) = -\infty$$

例如: $c3 = \begin{array}{|c|c|c|} \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \bullet & \bullet & \bullet \\ \hline \end{array}$

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索举例

等价的棋局，其估价值都是1 (5-4)

用黑实圈表示MAX，用圆圈代表MIN

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索举例

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

Max-Min搜索举例

则下一轮:

$c^{(0)} =$ (1)

人工智能学院

西安电子科技大学

XIDIAN UNIVERSITY

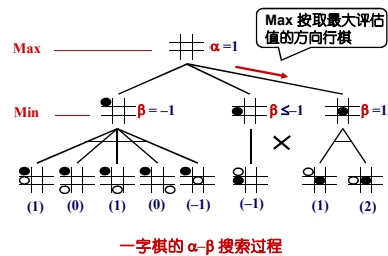
α - β 剪枝搜索

- 极大极小搜索过程由两个完全分离的两个步骤组成：
 - 第一、用宽度优先算法生成一棵博弈搜索树
 - 第二、估计值的倒推计算
- 缺点：这种分离使得搜索的效率比较低
- 改进： α - β 剪枝搜索
 - 边生成博弈树边计算评估各节点的倒推值
 - 理由：人类棋手的思维模式更多地表现出深度优先的特征，而不是宽度优先

人工智能学院

➤ 一字棋：

- 搜索策略: k-步博弈; 深度优先; 每次扩展一个节点; 一边扩展一边评估。
- 在 α - β 算法中:
 - 1) α : Max 节点估价值的下界 (或节点)
 - 2) β : Min 节点评价价值的上界 (与节点)



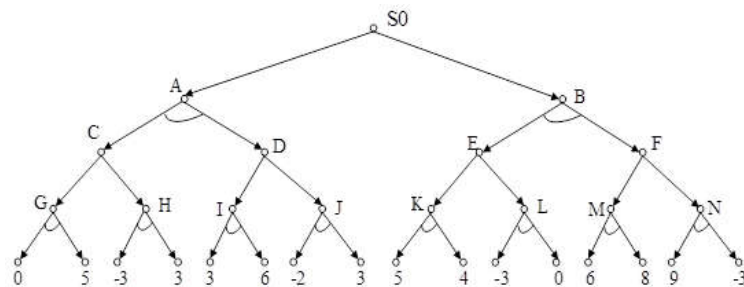
- 对于一个与节点来说，它取当前子节点中的最小倒推值作为它倒推值的上界，称此为 β 值；($\beta \leq \text{最小值}$)
- 对于一个或节点来说，它取当前子节点中的最大倒推值作为它倒推值的下界，称此为 α 值。($\alpha \geq \text{最大值}$)

规则一 (α 剪枝规则):

- 任何与节点 x 的 β 值如果不能升高其父节点的 α 值, 则对节点 x 以下的分支可停止搜索, 并使 x 的倒推值为 β

规则二(β 剪枝规则):

- 任何或节点 x 的 α 值如果不能降低其父节点的 β 值, 则对节点 x 以下的分支可停止搜索, 并使 x 的倒推值为 α



与/或树的搜索策略

- 盲目式搜索
 - 宽度优先搜索
 - 深度优先搜索
- 博弈树搜索
 - ▣ Max-Min搜索
 - ▣ α - β 剪枝

大作业一

题目：《基于图搜索算法的问题求解》

要求：采用一种基于状态空间表示的搜索算法（如宽度优先搜索，深度优先搜索，有序搜索，A*算法等）编程解决一种智力游戏或智力问题（如迷宫问题，数码难题，寻路问题等），或采用一种博弈树搜索算法（如Max-Min搜索， α - β 剪枝等）编程实现一种棋类游戏（如一字棋，五子棋等）。采用C、Python或Matlab编程，进行仿真结果分析，并撰写小论文。

提交时间：11.17日